

CSC270 · DATABASE SYSTEMS · LECTURE 28

Scaling Beyond One Server

MongoDB Sharding and Replication

CSC270 · Database Systems — COMSATS University Islamabad, Lahore Campus

Agenda

- 1 Why replication exists: availability and read scaling
- 2 Replica sets — primary and secondary roles
- 3 Automatic failover and elections
- 4 How reads and writes are routed across a replica set
- 5 Why sharding exists: write and storage scaling
- 6 Sharded cluster architecture: shards, config servers, mongos
- 7 Choosing a good shard key, and the hotspots a bad one causes
- 8 The balancer, and why production clusters use both together

Recap: One Server Is Not Enough

- Replication copies the same data to a replica set — solving availability and read scaling.
- Writes always go to the primary; reads can go to secondaries, at the cost of possible staleness.
- Sharding splits different slices of a collection across shards — solving write and storage capacity.
- Every query enters through mongos, which uses config-server metadata to route it.
- The shard key decides whether capacity really scales.
- Production clusters use both techniques together.

Why Replication Exists

A single mongod, however well it handles documents, is still just one server — one point of failure, and one machine's worth of disk and CPU. Replication keeps multiple synced copies of the same data to solve two problems:

● High availability

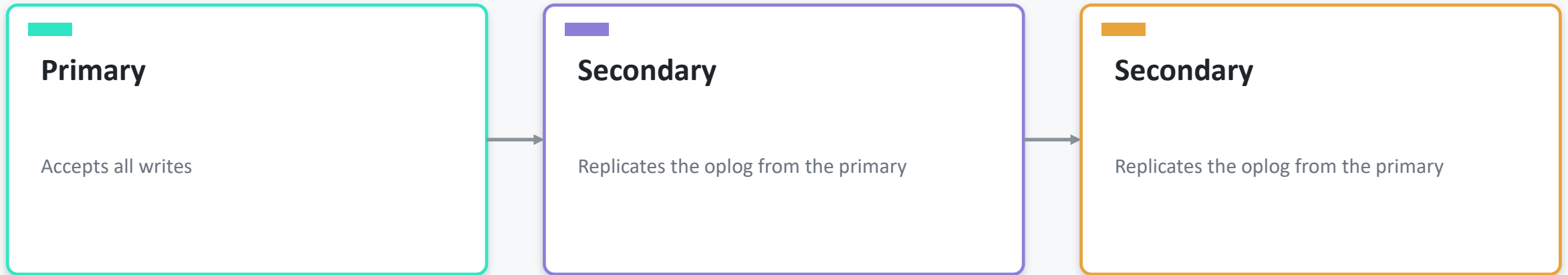
If the one server holding your data crashes, the application goes down with it. Multiple copies mean it can keep running on a surviving copy.

● Read scaling

A single server has a ceiling on reads per second it can serve; spreading reads across several copies raises that ceiling.

Replica Sets

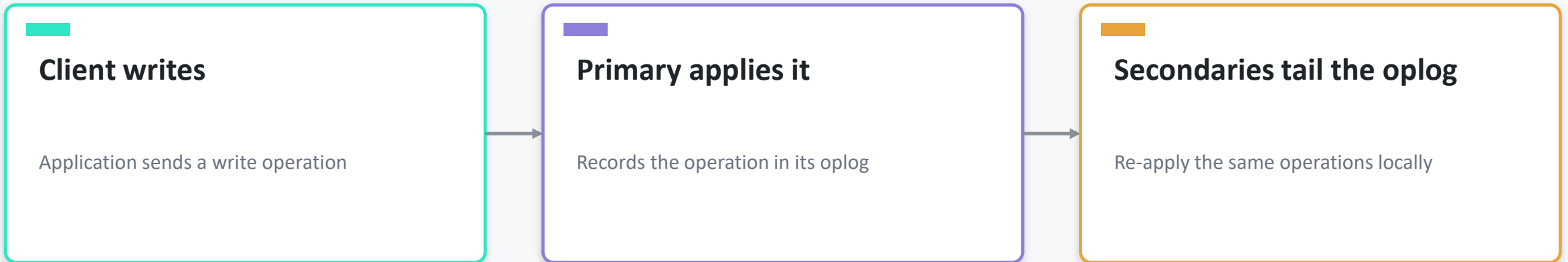
A replica set is a group of mongod instances that all hold the same data. It almost always has an odd number of members (three is typical) — which matters for elections below.



A THREE-MEMBER REPLICA SET

Exactly one member is primary at any moment — the only node that accepts writes. Every other member is a secondary, continuously replicating the primary's changes by tailing its oplog (operations log) and re-applying the same operations locally.

Primary and Secondary Nodes



SECONDARIES ARE ALWAYS SLIGHTLY BEHIND

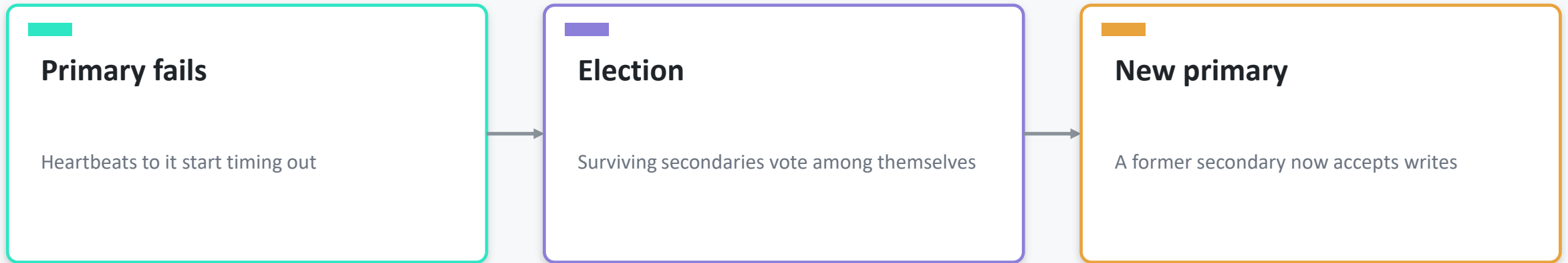
Usually by milliseconds, but under heavy load or a slow network link that lag can grow — which matters for the read-routing choice ahead.

THE OLD PRIMARY REJOINS AS A SECONDARY

Once a failed primary comes back online, it does not reclaim its role automatically — it rejoins as a secondary, catches up from the oplog, and needs a fresh election to become primary again.

Automatic Failover

If the primary becomes unreachable — a crash, a network partition, planned maintenance — the remaining members detect this through heartbeats and trigger an election.



WHY AN ODD MEMBER COUNT

A majority (quorum) of the replica set's members must be reachable and agree for an election to succeed. With three members, losing any one still leaves a clear majority of two able to elect a primary; with an even count, a split can leave neither side with a majority.

Failover, Step by Step: Three-Member Replica Set

Members P (primary), S1 and S2 (secondaries). At 10:00, P crashes. Here is what happens next.

● Step 1 — Heartbeats time out

S1 and S2 stop hearing from P and mark it unreachable within seconds.

● Step 2 — Election

S1 and S2 vote. Two of three votes is a majority, so the election succeeds.

Read and Write Operations in Replication

Writes always go to the primary — the only member allowed to accept them. Reads can be directed to secondaries too, controlled by the application's read preference:

Read preference	Behavior
primary (default)	All reads go to the primary — always the most up-to-date data
primaryPreferred	Prefer the primary, fall back to a secondary if unreachable
secondary	All reads go to a secondary — spreads out load, may return stale data
secondaryPreferred	Prefer a secondary, fall back to the primary if none available
nearest	Route to whichever member has the lowest network latency

A TRADE-OFF, NOT A FREE WIN

Routing reads to secondaries raises read capacity, but oplog replication delay means an app reading a secondary can see slightly stale data — fine for a dashboard, usually unacceptable for a “did my write succeed” confirmation.

Why Sharding Exists

Replication solves availability and read capacity — but every member of a replica set still holds the entire dataset, so it does nothing for a dataset too large, or a write volume too high, for any single server to hold or handle at all.

SHARDING PARTITIONS, REPLICATION COPIES

Sharding solves a different problem: it partitions a collection's data across multiple servers, so each one holds only a slice of the total — both storage and write capacity grow as you add more shards.

THE APPLICATION NEVER TALKS TO SHARDS DIRECTLY

A sharded MongoDB deployment adds a routing layer and a metadata layer on top of the shards themselves — covered on the following slides.

Sharded Cluster Architecture



Application

Sends every query exactly as it would to a single mongod



mongos (query router)

Figures out which shard(s) hold the relevant data, consulting config server metadata



Shard 1

Holds one slice of the data — e.g. regNo A–H



Shard 2

Holds another slice — e.g. regNo I–P



Shard 3

Holds another slice — e.g. regNo Q–Z



CONFIG SERVERS

Store the cluster's metadata — which chunks (ranges of shard-key values) currently live on which shard — and are themselves run as a replica set for availability. Each shard, in production, is not a single server but its own replica set.

Shard Keys

Every sharded collection needs a shard key — the field MongoDB uses to decide which chunk, and shard, a document belongs to:

● High cardinality

Many distinct values, so data can split into many chunks. Five-value fields never produce more than five useful chunks.

● Even distribution

Values spread roughly evenly across the range, so no one chunk (or shard) ends up disproportionately large or busy.

● Avoiding hotspots

A monotonically increasing key sends every new write to the shard owning the highest-value chunk — a bottleneck.

■ A TIMESTAMP OR AUTO-INCREMENT ID IS A CLASSIC BAD SHARD KEY

It looks high-cardinality, but every new document has a larger value than the last, so all new writes land on the single shard holding the topmost chunk — exactly the hotspot sharding was supposed to prevent. A compound shard key with a higher-entropy field is a better choice.

Choosing a Shard Key: studentId vs. Enrollment Year

Same students collection, three shards. Compare two candidate shard keys on the three criteria.

● High cardinality

studentId has ~100,000 values, so many chunks. Enrollment year has 4 values, so few chunks.

● Even distribution

studentId spreads evenly across the range. Enrollment year puts whole cohorts on one shard.

● Avoiding hotspots

A sequential numeric ID hot-spots the top chunk; a hashed studentId spreads new inserts.

■ VERDICT

Hashed studentId is the better key. Enrollment year has too few values and clusters new writes.

Query Routing and mongos

TARGETED QUERY

Filter includes the shard key

mongos computes exactly which shard(s) hold the answer



Forwards only there

Cheap — touches only the relevant shard(s)

SCATTER-GATHER QUERY

Filter omits the shard key

mongos can't tell which shard has the answer



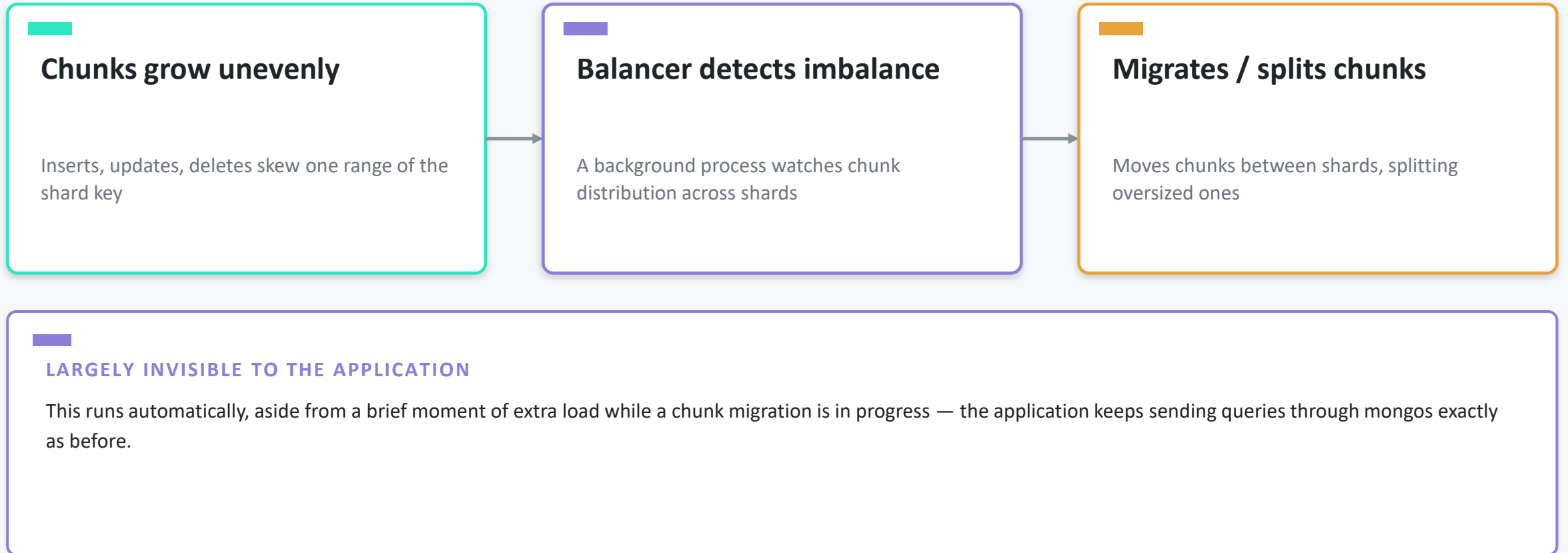
Sends to every shard, merges results

Far more expensive

KNOW YOUR WORKLOAD FIRST

Shard-key choice should be driven by the application's most common queries — the same “know your workload” principle behind Lecture 25's index selection. A poorly chosen key turns routine queries into expensive scatter-gathers.

Balancing and Data Distribution



Targeted vs. Scatter-Gather Queries

Targeted query

`find({ studentId: 1234 })` goes only to the shard that owns that chunk.

Scatter-gather query

`find({ gpa: { $gt: 3.5 } })` makes mongos ask every shard, then merge the results.

Why the difference

Targeted reads touch one shard; scatter-gather cost grows with the number of shards.

KEY TAKEAWAY

Filter on the shard key whenever you can. Otherwise every shard does work for one answer.

Sharding vs. Replication

Aspect	Replication	Sharding
Problem solved	High availability, read scaling	Horizontal write and storage scaling
What each node holds	The same complete dataset	A different slice of the dataset
Failure it protects against	Losing a single server	Running out of capacity on a single server
Key components	Primary, secondaries	Shards, config servers, mongos
Typical unit	A replica set	A sharded cluster (built from multiple replica sets)

NOT COMPETING TECHNIQUES

They answer different questions, and a serious production deployment uses both simultaneously — shown on the next slide.

Sizing a Cluster: When One Replica Set Is Not Enough

Illustrative numbers: 2 TB of data today, 4 TB disks per server, and a peak of 15,000 writes/s against a single primary's 10,000 writes/s limit.



STEP 1: STORAGE

2 TB fits on one server today, but 10 TB of growth needs at least three shards of data.



STEP 2: WRITES

15,000 writes/s exceed one primary, so split the writes across two or more shards.

A Realistic Production Topology



mongos query router(s)

What the application actually connects to



Config server replica set

Cluster metadata, itself replicated for availability



Shard 1 = its own replica set

Primary + secondaries, holding one slice of the data



Shard 2 = its own replica set

Primary + secondaries, holding another slice

Key Takeaways

- Replication copies the same data across a replica set — solving availability and read scaling. Writes go to the primary; reads can route to secondaries via a read preference.
- Automatic failover elects a new primary from surviving secondaries, requiring a majority (quorum) to agree.
- Sharding splits different slices of a collection across shards — solving horizontal write and storage capacity, a different problem from replication.
- A sharded cluster routes every query through mongos, using config-server metadata to send a targeted query or an expensive scatter-gather.
- Shard key choice — high cardinality, even distribution, no monotonic hotspot — is the single most consequential sharding decision, echoing Lecture 25's index selection.
- In production, both techniques combine: a sharded cluster's individual shards are themselves replica sets.

Course Close: The Recurring Trade-Offs

- Consistency versus availability: CAP forces a choice during a partition.
- Normalization versus embedding: model for the read pattern, not only the shape of the data.
- One server versus many: vertical scaling has a ceiling; horizontal scaling adds coordination cost.
- Index read speed versus write cost: every index slows writes.
- Shard key choice decides whether capacity actually scales.
- Every design is a trade-off — name what you gave up.